Local Variable:	name fish_and_chips x_axis thx1138 _x _26
Instance Variable:	@name @point_1 @X @_ @plan9
Class Variable:	@@total @@symtab @@N @@x_pos @@SINGLE
Global Variable:	$debug $CUSTOMER $_ $plan9 $Global
Class Name:	String ActiveRecord MyClass
Constant Name:	FEET_PER_MILE DEBUG

Table 1—Example variable, class, and constant names

Ruby hash syntax is similar to array syntax. A hash literal uses braces rather than square brackets. The literal must supply two objects for every entry: one for the key, the other for the value. Most generically, the key and value are separated by =>, though we'll see that there is a very commonly used shortcut.

For example, you could use a hash to map musical instruments to their orchestral sections.

```
instrument_section = {
  "cello" => "string",
  "clarinet" => "woodwind",
  "drum" => "percussion",
  "oboe" => "woodwind",
  "trumpet" => "brass",
  "violin" => "string"
}
```

The thing to the left of the => is the key, and the thing to the right is the corresponding value. Keys in a particular hash must be unique; you can't have two entries for "drum." The keys and values in a hash can be arbitrary objects. You can have hashes where the values are arrays, other hashes, and so on. The order of the keys in the hash is stable and will always match the order in which the keys were added to the hash. If you assign a new value to a key, the old value is erased.

Hashes are indexed using the same square bracket notation as arrays.

```
p instrument_section["oboe"]     # => "woodwind"\n"string"
p instrument_section["cello"]    # => "string"\nnil
p instrument_section["bassoon"]  # => nil
```

The default behavior of a hash when indexed by a key it doesn't contain is to return nil, representing the absence of a value.

Sometimes you'll want to change this default behavior. For example, if you're using a hash to count the number of times each different word occurs in a file, it's convenient to have the default value be zero. Then you can use the word as the key and simply increment the corresponding hash value without worrying about whether you've seen that word before. This can be done by specifying a default value when you create a new, empty hash:

```
histogram = Hash.new(0)    # The default value is zero
histogram["ruby"] # => 0
histogram["ruby"] = histogram["ruby"] + 1
histogram["ruby"] # => 1
```